

Phase Task Mutation Performance Audit

Read-only audit · High-frequency superintendent task mutations · Generated from repository analysis · Do not modify code based on this document alone without an implementation plan

Scope: Complete, Start, Mark N/A, Schedule, Reschedule.

Not in scope: code changes, Transaction Engine migrations, caching, optimistic UI implementation.

Verdict: READY TO OPTIMIZE

1. Executive Summary

Superintendent task mutations in Phase are **legacy online REST** → **Prisma** paths. **None** of the five audited workflows use the Transaction Engine (only punch-item create does).

The dominant perceived-latency pattern on House Details:

- 1. **User waits for the mutation API** before most UI updates (inline complete is *post-response*, not true optimistic).
- 2. **Every successful mutation triggers** `GET /api/homes/[id]/forecast`, which recomputes CPM, persists forecast fields on every task, and returns the full home + all tasks payload.
- 3. **Gates are refetched** in parallel (`GET /api/homes/[id]/gates`), loading all home tasks again and running N punch counts.
- 4. `setHome(fullHome)` **rerenders the entire ~1,250-line** `HomeDetailPage`, including every category accordion and every work-item card.

Heaviest server paths: Schedule (duplicate full-home task loads for guards), Mark N/A (forecast persist on server plus client forecast refetch), Reschedule with SMS (Twilio in-request).

Best existing pattern: Inline complete applies a local task patch immediately *after* PATCH returns, then background-refetches forecast/gates — but the button still shows “Saving...” until PATCH completes.

2. Five Workflow Maps

Action	Client entry	API	Method	Primary DB mutations	Post-mutati client
Complete	<code>HomeDetailPage.handleMarkCompleted</code>	<code>/api/tasks/[id]</code>	PATCH	<code>homeTask.update</code> ; optional <code>contractorAssignment.upsert</code> ; <code>auditLog</code> ; <code>recalculateHomeCompletion</code>	After PATCH patch one t play sound; background /forecast + ,
Complete	<code>TaskModal.handleStatusChange("Completed")</code>	same	PATCH	same	Waits PATCH- setCurrentTe onUpdate – forecast + g closes mode
Start	<code>TaskActionSheet.handleStart (Flow)</code>	<code>/api/tasks/[id]/start</code>	POST	<code>homeTask.update</code> ; <code>activityEvent.create</code>	Waits POST; onSuccess – Flow refetch
Start	<code>TaskModal.handleStatusChange("InProgress")</code>	<code>/api/tasks/[id]</code>	PATCH	Same as complete + audit + recal; no task_started activity	Waits PATCH- forecast + g
Mark N/A	<code>MarkNotApplicableDialog.handleSubmit</code>	<code>/api/tasks/[id]/not-applicable</code>	POST	<code>homeTask.update</code> ; audit; activity; recal; computeHomeForecastAndPersist	Waits POST; handleTaskL → another /forecast + ,

Action	Client entry	API	Method	Primary DB mutations	Post-mutati client
Schedule	TaskModal.handleSave, Flow schedule	/api/tasks/[id]	PATCH	homeTask.update; category/gate guards; audit; recalc; task_scheduled activity	Waits PATCH- forecast + g modal may : open for SM
Reschedule	RescheduleTaskDialog.submit	/api/tasks/[id]/reschedule	POST	history.create + homeTask.update (tx); audit; activity; optional sync SMS	Waits POST; forecast + g + history ref

Note: TaskModal.handleSave on an already-scheduled date change can PATCH without history — proper reschedule is only via RescheduleTaskDialog. Handlers live under app/api/tasks/[id]/. No TaskCard component; tasks are inline Cards in HomeDetailPage.

3. Client-Side Waiting Analysis

Action	Waits for fetch?	Optimistic?	Loading UX	Refetch
Complete (inline)	Yes — PATCH first	No (local patch is post-response)	Per-card "Saving..."	Forecast + gates (background)
Complete (modal)	Yes	No	Modal loading	Forecast + gates; closes modal
Start (Flow)	Yes	No	Sheet spinner	Full /api/flow
Start (modal)	Yes	No	Modal loading	Forecast + gates
Mark N/A	Yes	No	Dialog Loader2	Forecast + gates (double forecast)
Schedule	Yes	No	Modal/sheet loading	Forecast + gates
Reschedule	Yes	No	Dialog loading	Forecast + gates + history

No router.refresh() on House Details task mutations. No SWR/React Query — manual fetch + useState(home).

Perceived latency drivers: mutation RTT; no status change until response; forecast refetch often slower than mutation; full-page rerender after each setHome.

4. Refetch / Refresh Analysis

Mutation path	Data refetched	Scope	Necessary?
All via handleTaskUpdate / inline follow-up	GET /api/homes/[id]/forecast	Full home + all tasks; server CPM + N task updates	Partially — not needed before showing task status
Same	GET /api/homes/[id]/gates	All home tasks + per-gate punch counts	Unclear — only if unlock state changed
Reschedule	reschedule history	History list	Yes for reschedule
Mark N/A	Server computeHomeForecastAndPersist in POST	Same as forecast GET	Redundant with client forecast refetch
Flow start/schedule	GET /api/flow	Entire Flow queue	Necessary for Flow list
Dashboard / Calendar	Nothing	—	Stale until revisit / poll

Completing one task effectively refetches the entire home graph via forecast and replaces home state. Plans/thumbnail are not re-fetched, but full rerender still occurs.

5. Server Critical Paths

Complete — PATCH { status: "Completed" }

```

requireTenantContext      [REQUIRED BEFORE RESPONSE]
→ hasPermission(tasks:write) [REQUIRED]
→ homeTask.findFirst (+ home) [REQUIRED]
→ dependency guard (0-2 queries) [REQUIRED — correctness]
→ homeTask.update (+ includes) [REQUIRED]
→ contractorAssignment.upsert [COULD AFTER]
→ createAuditLog           [COULD AFTER]
→ recalculateHomeCompletion [COULD AFTER]
→ notifyTaskCompleted (no-op) [D]
→ return task JSON         [REQUIRED]

```

Start — POST /start

```

requireTenantContext → permission → findFirst → dep guard → update → createTaskStartedEvent → return

```

Activity is awaited today. No audit/recalc. PATCH InProgress path differs: audit + recalc, no task_started activity.

Mark N/A — POST /not-applicable

```

findFirst → update → audit → user.findUnique → activity → recalculateHomeCompletion
→ computeHomeForecastAndPersist → return

```

Forecast persist is the largest blocker — could be AFTER RESPONSE with immediate NA UI.

Schedule — PATCH with scheduledDate

```

requireTenantContext → billing gates → findFirst
→ homeTask.findMany (ALL tasks) [REQUIRED for category gates]
→ categoryGate.findMany + scan [REQUIRED]
→ checkGateBlocking (ALL tasks again + N) [REQUIRED]
→ update → upsert → audit → recalc → createTaskScheduledEvent (.catch) → return

```

Reschedule — POST /reschedule

```
findFirst → billing → gate check → $transaction(history + update)
→ findUniqueOrThrow → audit → notify (no-op)
→ optional sendTaskConfirmationInternal (SMS) [BLOCKS if resendSms]
→ activity → return { task, smsResent, warnings }
```

6. Database Query Analysis

Workflow	Approx. Prisma ops	Notable patterns
Complete	6–10	Sequential: read → deps → update → upsert → audit → recalc (4 queries)
Start (POST)	4–5	Lighter; no audit/recalc
Start (PATCH)	6–10	Same as complete
Mark N/A	15+	Forecast: load home/tasks, deps, phase data, \$transaction with N task updates + home
Schedule	12–20+	Duplicate homeTask.findMany; checkGateBlocking reloads all; per-gate punch counts
Reschedule	8–15+	Transaction OK; redundant re-read; SMS path adds reads + Twilio

N+1: getHomeGateStatus / checkGateBlocking run one punch count per critical gate. **Forecast GET:** access check → computeHomeForecastAndPersist again → full home.findUnique — triple work when N/A POST already persisted.

7. Auth / Tenant Cost

Every mutation calls `requireTenantContext()`:

- `getServerSession(authOptions)`
- Usually `prisma.user.findUnique` (+ company name)
- Impersonation path may add lookup

Then `hasPermission(role, "tasks:write")` — in-memory, cheap. Forecast refetch uses `requireTenantPermission("homes:read")` — session + user again + superintendent assignment check. No per-request caching of tenant context. Redundant but likely small vs forecast/gates.

8. Side-Effect Classification

Side effect	Complete	Start POST	Start PATCH	N/A	Schedule	Reschedule
Audit log	B	—	B	B	B	B
Activity	— (dead helper)	B	—	B	C/D	B
Notifications	D (no-op)	—	—	—	D	D
SMS	—	—	—	—	Separate send-confirmation	B if resendSms
Forecast persist	—	—	—	B	—	—
Home completion recalc	B	—	B	B	B	—

A = atomic with mutation · B = before response today · C = could after response · D = already async / no-op

9. React Rendering Analysis

Single client component `HomeDetailPage` (~1,248 lines). Tasks rendered inline as Cards inside category accordions — **no TaskCard**, **no React.memo**.

On any `setHome`:

- Recomputes `tasksByCategory`, progress bars, blocked reasons
- Rerenders all categories and all task cards
- Rerenders `HouseScheduleCard`, header, plans consumers
- `useSession()` on page — session updates also rerender whole tree

Estimate: Updating one `HomeTask` → entire House Details page rerenders.

10. Payload Size

Workflow	Request	Response
Complete	{ status: "Completed" } (~30 B)	Full <code>HomeTask</code> + contractor, home, subdivision (~2–5 KB+)
Start POST	empty	Same shape
N/A	{ reason, note? }	Full task + <code>templateItem</code> + NA metadata
Schedule	<code>scheduledDate</code> , <code>contractorId?</code> , <code>notes?</code> , <code>confirmManually?</code>	Full task graph
Reschedule	<code>scheduledDate</code> , <code>reason</code> , <code>note?</code> , <code>resendSms?</code>	{ task, smsResent, warnings }

Forecast GET returns entire home with all tasks — often **50–200+ KB JSON** on typical homes. Could theoretically return a delta DTO.

11. Transaction Engine Coverage

Workflow	Engine?	Migration would require
Complete	Legacy	New transaction type, idempotent apply, client queue + reconcile, dependency validation
Start	Legacy	Same; unify POST /start and PATCH <code>InProgress</code>
N/A	Legacy	Forecast side-effect policy (sync vs deferred)

Workflow	Engine?	Migration would require
Schedule	Legacy	Gate/category guard snapshots or revalidation on apply
Reschedule	Legacy	History + confirmation demotion + optional SMS outbox

Existing TE: punch create only (lib/transactions/, TransactionEngineProvider, IndexedDB queue).

12. Offline / Conflict Implications

Workflow	Classification
Complete	Optimistic candidate; conflict-sensitive (dependency guard, 409)
Start	Optimistic candidate; dep guard; POST /start less strict than PATCH
Mark N/A	Needs server confirmation for forecast/completion; reversible via mark-applicable
Schedule	Conflict-sensitive (gates, category gates, billing); SMS external
Reschedule	Most conflict-sensitive (history, Confirmed→Scheduled demotion, SMS)

13. Existing Instrumentation

Present (temporary/debug):

- Date.now() + localhost ingest in handleMarkCompleted
- debugLog + ingest in PATCH /api/tasks/[id], GET /forecast, GET /gates

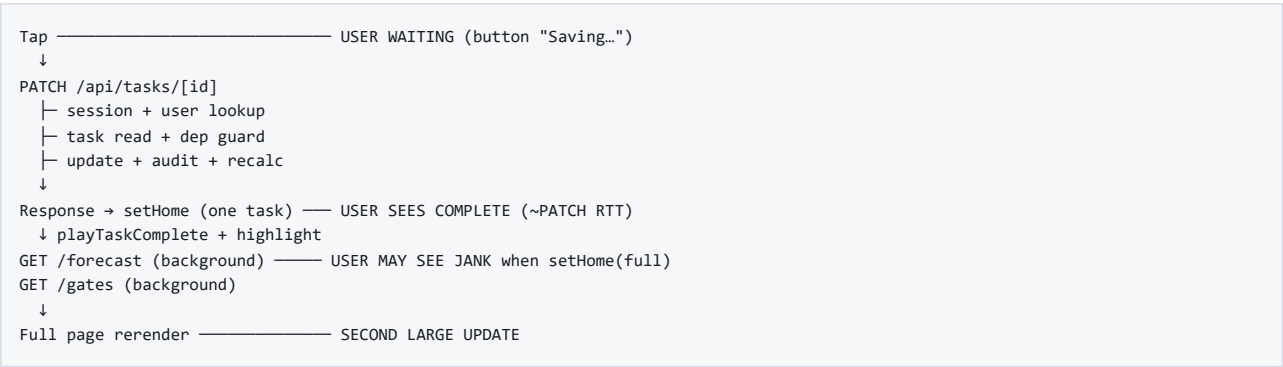
Absent: performance.now() / Web Vitals / OpenTelemetry; Prisma query logging; Server-Timing; production T0–T6 marks.

Proposed future measurement:

- Client: tap → local patch → fetch start/end → reconcile
- Server: auth, guards, mutation, side-effects
- Log P50/P95 for PATCH vs forecast vs gates separately

14. Current Latency Models

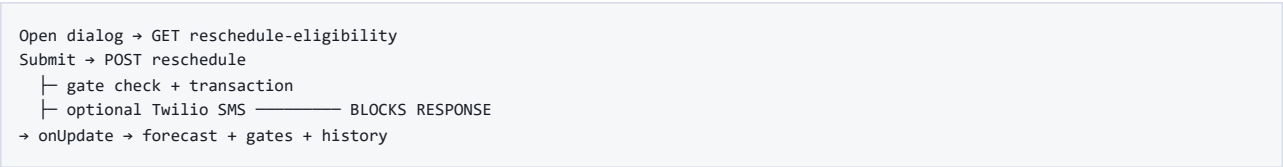
Complete (inline — primary superintendent path)



Mark N/A (worst case)



Reschedule (+ SMS)



15. Top 10 Bottlenecks (Ranked)

#	Bottleneck	Impact	Freq	Effort	Risk
1	Post-mutation GET /forecast (full CPM + persist + huge payload)	Very high	Every mutation	Medium	Medium

#	Bottleneck	Impact	Freq	Effort	Risk
2	No true optimistic UI — wait for API before status change	Very high	Complete/start/schedule/NA	Medium	Medium
3	setHome(full) → entire page rerender	High	Every mutation	Medium	Low
4	N/A double forecast (server persist + client refetch)	High	N/A	Low	Low
5	Schedule guard queries (2× full home task load + gate counts)	High	Schedule/reschedule	Medium	Medium
6	Sync side effects before response (audit, recalc, activity, forecast on N/A)	Medium	All	Low–Med	Low
7	Reschedule SMS in-request	Medium	Reschedule w/ SMS	Medium	Medium
8	Large mutation responses (full includes)	Medium	All	Low	Low
9	Gates refetch after every mutation	Medium	Most	Low	Low
10	Flow full refetch on start/schedule	Medium	Flow users	Low	Low

16. Recommended Optimization Sequence

Phase P1 — Client refetch/rerender (safest wins)

- Patch local home.tasks from mutation response; stop awaiting forecast for status display
- Defer /forecast and /gates (idle / background, coalesce rapid mutations)
- Remove duplicate forecast on N/A (server OR client, not both)
- Narrow rerender: split task list state or memoize rows carefully

Phase P2 — True optimistic UI

- Complete/start: apply status on tap; rollback on 409/400
- Keep dependency errors as server source of truth

Phase P3 — Server critical path

- Move audit/activity/recalc to async jobs or waitUntil
- Collapse schedule guard reads (single task load)
- Slim PATCH response (TaskMutationResult DTO)

Phase P4 — Transaction Engine for status mutations

- Complete/start/N/A first (highest frequency)

Phase P5 — Schedule/reschedule

- Outbox SMS; optional optimistic date with server validation

17. Expected Perceived Improvements

Change	Expected effect
Optimistic status on tap	100–300ms perceived → near <100ms visual
Defer forecast/gates	Removes 500ms–2s+ blocking tail on most taps
Stop full-page rerender	Smoother UI; less main-thread work on large homes
N/A single forecast	~30–50% faster N/A server + client combined
Slim responses	Faster JSON parse + less rerender cost

18. Risks / Correctness Guardrails

Do not optimize away without replacement:

- Tenant scoping on every mutation
- Dependency guard (409 DEPENDENCY_BLOCKED)
- Category/punch gate checks on schedule/reschedule
- Reschedule history immutability
- Confirmed → Scheduled demotion on reschedule
- Contractor confirmation / SMS semantics
- Audit/activity (can defer, not drop)
- Forecast accuracy for portfolio/Dashboard (eventual consistency OK if labeled)

Risky ideas: optimistic schedule without gate revalidation; skipping forecast forever; caching tenant context across users; batching mutations without version checks.

19. First Optimization to Implement

Defer and coalesce post-mutation /forecast + /gates refetches; patch task state immediately from the mutation response.

1. On PATCH/POST success, merge returned task into home.tasks (all entry points).
2. Fire /forecast + /gates in background; debounce if superintendent taps multiple tasks quickly.
3. For N/A, remove client forecast refetch (server already persists) or remove server persist from POST — pick one.
4. Add lightweight performance.mark for tap→patch→reconcile when implementing.

Final Status

READY TO OPTIMIZE

No fundamental architectural flaw blocks incremental performance work. The main issue is **synchronous request chains and full-home reconciliation**, not missing indexes or Transaction Engine absence (yet).

Target experience (evaluation only — not implemented): Tap → visible UI <100ms; durable local acceptance <150ms where TE applies; normal server acknowledgement ideally <500ms; user blocked waiting as close to 0ms as safely possible.